LAB TASK

Dynamic Programming: Implement 0/1 Knapsack problem

CODE:

```
#include <iostream>
#include <algorithm>
using namespace std;

int knapsack(int W, int wt[], int val[], int n)
{
    int dp[n + 1][W + 1];

    for (int i = 0; i <= n; i++)
    {
        for (int w = 0; w <= W; w++)
        {
            if (i == 0 || w == 0)
            {
                dp[i][w] = 0;
            }
            else if (wt[i - 1] <= w)
            {
                dp[i][w] = max(
                    val[i - 1] + dp[i - 1][w - wt[i - 1]],
                    dp[i - 1][w]
                );
            }
            else
            {
```

```
                dp[i][w] = dp[i - 1][w];
            }
        }
    }

    return dp[n][W];
}

int main()
{
    int n = 4;
    int W = 5;

    int wt[] = {2, 1, 3, 2};
    int val[] = {12, 10, 20, 15};

    cout << "Maximum value: "
         << knapsack(W, wt, val, n);

    return 0;
}
```

OUTPUT:

